

Lecture 09 : TCP and UDP (Part II)

Introduction

In the intricate world of computer networking, the transport layer plays a pivotal role in enabling seamless communication between applications running on different hosts. This lecture delves deeper into two fundamental protocols at this layer: Transmission Control Protocol (TCP) and User Datagram Protocol (UDP).

Understanding their internal mechanisms, particularly their header structures and connection management processes, is not merely an academic exercise; it's crucial for comprehending network performance, reliability, and, critically, potential vulnerabilities that can be exploited in network-based attacks.

This session will dissect the header fields of both TCP and UDP, revealing how they encapsulate essential information for data transfer. We will pay special attention to TCP's sophisticated connection establishment and termination procedures, known as handshakes, which are cornerstones of its reliability. Furthermore, we will explore how these very mechanisms can be manipulated, leading to significant security threats like Denial-of-Service (DoS) attacks. By the end of this discussion, you will have a clearer picture of how these protocols function at a granular level and their broader implications for network security and design.

Core Concepts

Term	Definition	Significance
TCP	Transmission Control Protocol, a connection-oriented, reliable transport layer protocol.	Ensures data delivery, order, and integrity, crucial for applications requiring high reliability.
UDP	User Datagram Protocol, a connectionless, unreliable transport layer protocol.	Offers speed and low overhead, suitable for applications where occasional data loss is acceptable.
Segment	The basic unit of data transfer at the TCP layer, comprising a header and application data.	Represents a stream of bytes broken into manageable chunks for

Term	Definition	Significance
		transmission over the network.
Sequence Number	A 32-bit field in the TCP header indicating the byte number of the first data byte in the segment.	Essential for ordering segments, detecting lost segments, and ensuring reliable, in-order delivery.
Acknowledgement Number	A 32-bit field in the TCP header indicating the next byte number the receiver expects.	Used by the receiver to confirm receipt of data and inform the sender which data can be purged from its buffer.
Window	A 16-bit field in the TCP header specifying the amount of data (in bytes) the receiver is willing to accept.	Controls the flow of data, preventing a fast sender from overwhelming a slow receiver (flow control).
3-Way Handshake	The process TCP uses to establish a connection between a client and a server.	Ensures both parties are ready to communicate and synchronize initial sequence numbers before data transfer.
4-Way Handshake	The process TCP uses to gracefully terminate an established connection.	Guarantees that all pending data has been sent and acknowledged by both sides before closing the connection.
Half-Open Connection	A state where a server has allocated resources for a TCP connection but the client has not completed the handshake.	A vulnerability that can be exploited in Denial-of-Service attacks by exhausting server resources.
Denial-of-Service (DoS) Attack	An attack where legitimate users are prevented from accessing a service due to	Highlights how protocol-level vulnerabilities can be

Term	Definition	Significance
	resource exhaustion or network flooding.	leveraged to disrupt network services.

Detailed Analysis

1. TCP Fundamentals and Operational Characteristics

TCP operates at the transport layer, providing a robust, host-to-host communication facility. Unlike the underlying IP protocol, which is unreliable and connectionless, TCP offers a connection-oriented and reliable service to applications. This reliability is achieved through various mechanisms, including error control and retransmission.

Key features of TCP include:

- * **Host-to-Host Communication:** Enables processes on different machines (local and remote) to communicate.

- * **Stream Delivery Service:** Data is treated as a continuous stream of bytes, broken into segments for transmission and reassembled at the destination. This is a stream-oriented service.

- * **Full Duplex Communication:** Allows simultaneous two-way data flow between the communicating hosts.

- * **Multiplexing and Demultiplexing:** Supports multiple active connections over the same logical links, allowing different applications to share the network infrastructure.

- * **Connection-Oriented and Reliable Service:** Establishes a dedicated logical connection before data transfer and ensures all data arrives correctly and in order.

Despite providing a reliable service, TCP relies on the unreliable datagram service of IP. This necessitates significant internal management by TCP, including bookkeeping, buffering, and error correction, to present a reliable interface to higher-layer applications.

Conceptually, TCP establishes a logical connection, often visualized as a "pipe," between a sending and a receiving process. Both the sender and receiver maintain buffers to manage the stream of bytes. The sender's buffer holds data to be sent (some already sent, some not yet), while the receiver's buffer stores incoming data (some received but not yet read by the application, some still awaited). TCP automatically manages these buffers, including requesting retransmission of missing bytes to ensure correct order delivery. The basic unit of data transfer at the TCP level is called a `segment`, which is a stream of bytes.

2. TCP Header Format

The TCP header is a critical component, typically ranging from a minimum of 20 bytes to a maximum of 60 bytes, with each row representing a 32-bit (4-byte) quantity.

- * **Source Port Number (16 bits):** Identifies the sending application process.
- * **Destination Port Number (16 bits):** Identifies the receiving application process.
- * **Sequence Number (32 bits):** Indicates the byte number of the first data byte in the current segment. This is crucial for tracking the order of bytes in the data stream and ensuring reliability.
- * **Acknowledgement Number (32 bits):** Sent by the receiver to inform the sender of the next byte number it expects to receive. This confirms successful receipt of all bytes up to the acknowledged number. For example, if the receiver has received up to byte 1000, it will send an ACK of 1001.
- * **Header Length (4 bits):** Specifies the length of the TCP header in 32-bit words. The minimum value is 5 (for a 20-byte header).
- * **Reserved (6 bits):** Reserved for future use.
- * **Flags (6 bits):** A set of control bits that manage the connection state and data flow:
 - * **URG (Urgent Pointer Active):** If set to 1, indicates that the Urgent Pointer field is significant.
 - * **ACK (Acknowledgement Valid):** If set to 1, indicates that the Acknowledgement Number field is significant.
 - * **PSH (Push Function):** If set to 1, requests the receiving application to immediately push the data to the application, indicating the end of a message.
 - * **RST (Reset Connection):** If set to 1, resets the connection, often due to an error or to reject a connection attempt.
 - * **SYN (Synchronize Sequence Numbers):** If set to 1, initiates a connection. Used during the 3-way handshake.
 - * **FIN (No More Data from Sender):** If set to 1, indicates that the sender has no more data to transmit and wishes to terminate the connection.
- * **Window (16 bits):** Specifies the size of the receive window, indicating how many bytes the receiver is currently willing to accept beyond the acknowledged byte. This is vital for flow control, allowing the receiver to regulate the sender's transmission rate. This is part of the sliding window protocol.
- * **Checksum (16 bits):** Used for error detection across the entire TCP segment (header + data) and a pseudo-header. It uses the same 1's complement sum algorithm as the IP checksum.
- * **Urgent Pointer (16 bits):** If the URG flag is set, this field points to the last byte of urgent data within the segment.

* **Options (Variable):** An optional field that can extend the TCP header for various purposes, such as specifying Maximum Segment Size (MSS).

The TCP checksum calculation includes a "pseudo-header" which borrows fields from the IP layer (Source IP, Destination IP, Protocol, TCP Length). This provides an additional layer of checking to ensure the segment reaches the correct destination IP address and protocol.

3. TCP Connection Establishment: The 3-Way Handshake

Establishing a TCP connection involves a three-step process known as the 3-way handshake, ensuring both client and server are ready to communicate and synchronize their initial sequence numbers.

1. **Client initiates (SYN):** The client sends a TCP segment with the `SYN` flag set to 1 and an initial sequence number (e.g., `SEQ = 100`). This is a connection request.
2. **Server acknowledges and synchronizes (SYN-ACK):** Upon receiving the SYN request, the server allocates resources for the connection and responds with a segment where both `SYN` and `ACK` flags are set to 1. It includes its own initial sequence number (e.g., `SEQ = 200`) and acknowledges the client's SYN by setting the `ACK` number to the client's sequence number plus one (e.g., `ACK = 101`).
3. **Client acknowledges (ACK):** The client receives the SYN-ACK, confirms the server's readiness, and sends a final segment with the `ACK` flag set to 1. This segment acknowledges the server's SYN by setting the `ACK` number to the server's sequence number plus one (e.g., `ACK = 201`). It also includes its own sequence number (e.g., `SEQ = 101`).

Once the server receives this final ACK, the connection is fully established, and data transfer can begin.

4. TCP Connection Termination: The 4-Way Handshake

Terminating a TCP connection is a more involved four-step process, ensuring that all data has been exchanged and acknowledged before the connection is fully closed.

1. **Client initiates termination (FIN):** When the client wishes to close the connection, it sends a segment with the `FIN` flag set to 1.
2. **Server acknowledges FIN (ACK):** The server receives the client's FIN and responds with an `ACK` segment, acknowledging the client's termination request. At this point, the server can still send data to the client (half-close state).
3. **Server initiates termination (FIN):** Once the server has no more data to send, it also sends a segment with the `FIN` flag set to 1 to the client.
4. **Client acknowledges FIN (ACK):** The client receives the server's FIN and responds with a final `ACK` segment, acknowledging the server's termination.

request. After this, the client waits for a period (TIME_WAIT state) to ensure the ACK is received by the server, and then the connection is fully closed.

This 4-way handshake ensures a graceful shutdown, where both sides can finish sending any remaining data before closing.

5. Security Implications: Half-Open Connections and DoS Attacks

The TCP 3-way handshake, while robust, introduces a vulnerability known as a "half-open connection." This occurs when a client sends a SYN request, and the server responds with a SYN-ACK, but the client either deliberately or accidentally fails to send the final ACK.

- * **Half-Open State:** The server, having received the initial SYN, allocates resources (e.g., buffer space, table entries) and waits for the client's final ACK. If this ACK never arrives, the server's resources remain tied up indefinitely (or until a timeout).

- * **Denial-of-Service (DoS) Attack:** Malicious actors can exploit this by sending a large number of SYN requests to a target server but never completing the 3-way handshake (e.g., by spoofing source IP addresses or simply not sending the final ACK). Each incomplete connection consumes server resources. If enough such requests are sent, the server's connection table fills up, preventing legitimate users from establishing new connections. This effectively denies service to legitimate users, hence the name Denial-of-Service attack. While timeout mechanisms exist to eventually clear these half-open connections, a continuous flood of SYN requests can overwhelm the server, making it unresponsive.

6. UDP Header Format and Characteristics

In contrast to TCP, UDP is a much simpler, connectionless, and unreliable protocol. Its header reflects this simplicity, resulting in lower overhead.

- * **Source Port Number (16 bits):** Identifies the sending application process.

- * **Destination Port Number (16 bits):** Identifies the receiving application process.

- * **Length (16 bits):** Specifies the total length of the UDP header and data in bytes.

- * **Checksum (16 bits):** An optional field for error detection. If not used, it is set to zero. Like TCP, the UDP checksum also includes a pseudo-header for additional verification.

The total UDP header size is a fixed 8 bytes. Because UDP does not involve connection establishment (no 3-way handshake), flow control, or retransmission, it offers significantly faster data transmission. This makes UDP suitable for applications where speed is prioritized over absolute reliability, such as streaming media, online gaming, or DNS queries, where occasional packet loss is tolerable or handled by the application layer.

Key Takeaways

- [] TCP provides a reliable, connection-oriented, full-duplex, stream-oriented service at the transport layer.
- [] TCP segments are the basic data units, and both sender and receiver maintain buffers for data management.
- [] The TCP header includes crucial fields like Source/Destination Ports, Sequence Number, Acknowledgement Number, Window size for flow control, and various flags.
- [] TCP connection establishment uses a 3-way handshake (SYN, SYN-ACK, ACK) to synchronize communication.
- [] TCP connection termination uses a 4-way handshake (FIN, ACK, FIN, ACK) to gracefully close the connection.
- [] A "half-open connection" occurs when a TCP handshake is incomplete, which can be exploited in Denial-of-Service (DoS) attacks.
- [] UDP is a connectionless, unreliable protocol with a simpler 8-byte header, prioritizing speed over reliability.
- [] The choice between TCP and UDP depends on application requirements for reliability versus speed and overhead.

Conclusion

The detailed examination of TCP and UDP headers, along with their connection management processes, underscores the fundamental differences in their design philosophies and operational characteristics. TCP's complexity is a direct consequence of its commitment to reliability, ensuring that data arrives in order and without errors, a necessity for applications like web browsing and file transfer. In contrast, UDP's minimalist design prioritizes speed and efficiency, making it ideal for real-time applications where latency is critical and some data loss is acceptable.

Understanding the 3-way and 4-way handshakes of TCP is not just about knowing how connections are made and broken; it's also about recognizing the inherent vulnerabilities, such as the half-open connection state, that can be exploited for malicious purposes like Denial-of-Service attacks. This knowledge is paramount for network administrators and security professionals in designing resilient systems. As we move forward, the principles discussed here will serve as a foundation for exploring more advanced networking concepts, including how IP subnets are structured and managed, which will be the focus of our next lecture.